The **grammar** and **parser** commands

Basil Fürer Dmitriy Traytel

October 28, 2026

Abstract

A definitional package for indentation-sensitive, context-free grammars in Isabelle/HOL. **grammar** elaborates a grammar specification with semantic actions into a grammar constant, per-nonterminal decoders and a mutual inductive relation. **parser** emits an executable parser for it together with soundness and completeness theorems. The parser generator backends are extensible; this package provides an LALR(1)-style backend with layout-aware lookahead out of the box.

Contents

1 Overview **1**

2 Grammar specifications **2**
2.1 Meaning of the annotations 4
2.2 What is defined 5

3 Parser generation **5**

4 LR parser **6**

5 Example **8**

1 Overview

The package defines grammars and parsers for indentation-sensitive context-free grammars in the sense of Adams [1]. A grammar is a value of type (*'n*, *'tok*) *is-cfg* (`Indentation_Sensitive.thy`): a list of productions over nonterminals *'n* and token classes *'tok*, in which every right-hand-side symbol carries an indentation relation *irel*. Parser input is a list of token records (*'tok*, *'ch*, *'m*) *token-scheme* (`Token.thy`); the token class *tok* and the column *s-col* of the *span* are all the parser reads, the remaining fields and any record extension *'m* pass through to the semantic actions.

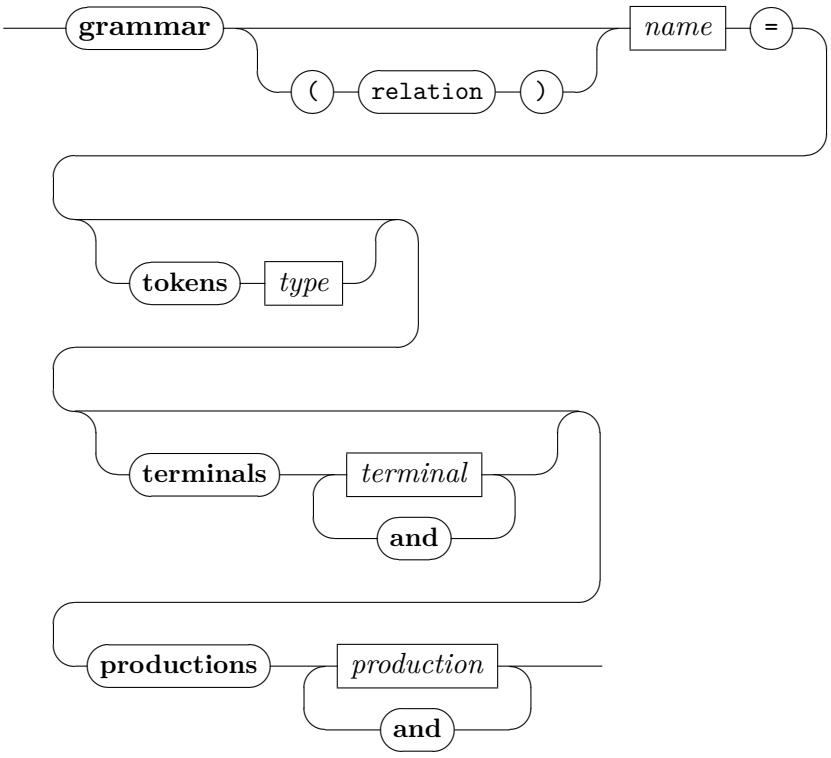
Two commands make this usable from Isar. **grammar** turns a grammar specification with semantic actions into a grammar constant and per non-

terminal decoders, optionally also into an inductive relation between words and values. **parser** emits an executable parser[1] for such a grammar together with soundness and completeness theorems.

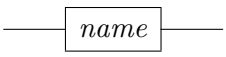
The parser generator is extensible; this package provides one backend, *LR*: an LALR(1)-style parser with layout constraints, running the paper's exact $k = 0$ machine under one token of lookahead propagated over the LR(0) automaton. Its determinism premise of completeness is decided by a static check (**by** *eval*).

2 Grammar specifications

grammar : *local-theory* → *local-theory*

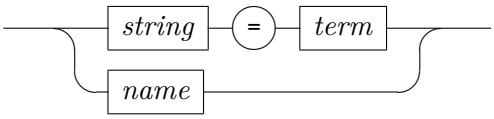


non-terminal

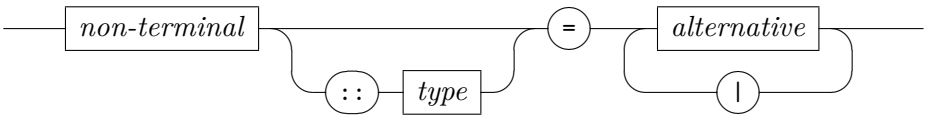


[1] Semantic actions must be executable if the parser is to be.

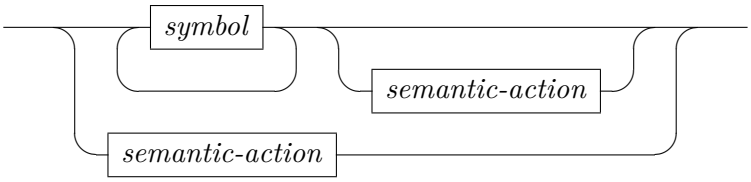
terminal



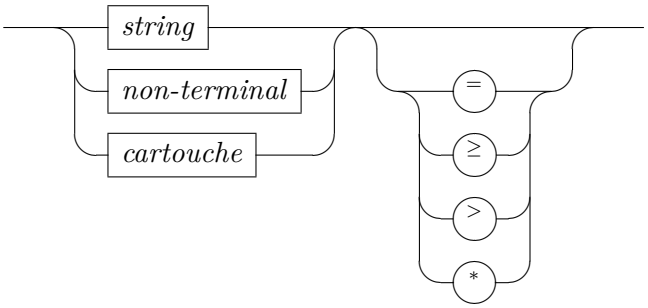
production



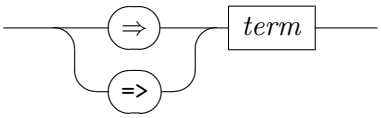
alternative



symbol



semantic-action



The categories *name*, *string*, *type*, *term* and *cartouche* are those of the Isar reference manual. The annotations may also be written in ASCII: `^=`, `^>=`, `^>` and `^*`.

grammar $G = \ldots$ checks the specification and defines the grammar constant G (**grammar** (*relation*) $G = \ldots$ also defines the relation family, see below).

The first non-terminal is the start symbol.

tokens T fixes the token record type. T is a ground instance of ($'tok$, $'ch$, $'m$) *token-scheme*: the plain record ($'tok$, $'ch$) *token*, or a client extension **record** *my-token* = (*my-tok*, *char*) *token* + ... (see below).

A type that is not a token record abbreviates (T, *unit*) *token*; without a **tokens** clause the token type is *string*.

terminals declares the terminal symbols. A *fixed spelling* $s = t$ lets the quoted spelling s stand for the token $t :: 'tok$ in the productions without binding it in semantic actions; a bare name declares a *token class*, used by its name in the productions with the whole token record as its value. A bare name that is neither a non-terminal nor declared is read as a term of type $'tok$ (for the *string* token type: the string itself).

productions lists the non-terminals. A group $A :: \tau = alt_1 \mid \ldots \mid alt_n$ declares the non-terminal A with *carrier* τ (the type of its semantic values and its alternatives). The ascription can be omitted when the carrier follows from the actions.

An alternative is a sequence of symbols, each with an optional annotation, followed by an optional semantic action. A symbol is a non-terminal, a fixed terminal, or an inline token $\langle t \rangle$ with $t :: 'tok$.

A non-terminal B contributes its value of type τ_B, and a token the complete token record.

The semantic action $\Rightarrow f$ computes the value of the alternative from the values of its value binding symbols, in order: $\tau_1 \Rightarrow \ldots \Rightarrow \tau_k \Rightarrow \tau_A$.

An alternative without action must have exactly one value symbol, whose value is passed on.

The annotation of a symbol is its indentation relation: $^{=}$ for *Eq*, $^{\geq}$ for *Ge*, $^{>}$ for *Gt* and * for *Any*, the default.

2.1 Meaning of the annotations

The indentation of a terminal is the column of its token, a nonterminal has no token of its own and its indentation is a free choice restricted only by the annotations.

A sentential form annotates every symbol with an indentation (column). Expanding $(A,\ i)$ by a production $A = {X_1}^{r1} \ \ldots \ {X_n}^{rn}$ chooses for each symbol an indentation j_k with *irel-sat* $r_k\ j_k\ i$:

$$\begin{aligned} \textit{irel-sat Eq } j\ i &\longleftrightarrow j = i \\ \textit{irel-sat Gt } j\ i &\longleftrightarrow i < j \\ \textit{irel-sat Ge } j\ i &\longleftrightarrow i \leq j \\ \textit{irel-sat Any } j\ i &\longleftrightarrow \textit{True} \end{aligned}$$

So $^{=}$ aligns a symbol with the nonterminal being expanded, $^{>}$ indents it further, $^{\geq}$ allows both, and * places no constraint. A block whose members are aligned is written $\textit{Block} = \textit{Stmt}^{=} \mid \textit{Block}^{=}\ \textit{Stmt}^{=}$: all statements share the indentation of the block. A grammar in which every annotation is * is an ordinary context-free grammar.

ilang G is the set of token words derivable in this way.

2.2 What is defined

For **grammar** G with non-terminals A of carrier τ_A and start symbol S:

The grammar constant G :: (*string*, *'tok*) *is-cfg* with its equation *G-def*. Nonterminals are represented by their names, the start symbol has indentation *0*.

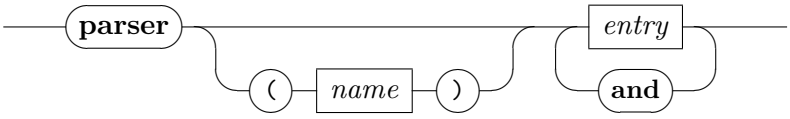
One decoder *G--dec-A* :: (*'m*, *'ch*, *string*, *'tok*) *ast* $\Rightarrow$ τ_A *option* per nonterminal, with the equations *G--dec-A.simps*. It evaluates the actions over a parse tree, matching each node against the productions of A. The decoders are defined together through *ast-rec*.

With (*relation*): the **inductive** family *G-A* :: *nat* $\Rightarrow$ *word* $\Rightarrow$ τ_A $\Rightarrow$ *bool*, where *G-A i w x* holds if the word w derives from (A, i) with value x, there is one introduction rule per alternative. The predicates are defined by one (mutual) inductive definition per strongly connected component of the nonterminals, lower components first. The facts are *G.intros*, *G.inducts* (the induction rule of each predicate, over its component) and *G.lang*: $(\exists\, x.$ *G-S 0 w x*$) \longleftrightarrow w \in$ *ilang G*.

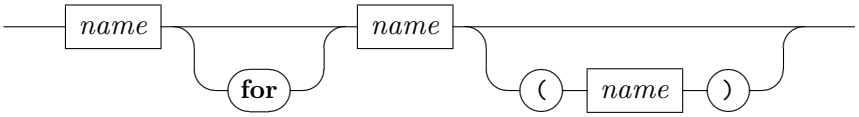
For a grammar without annotations the indentation argument is dropped: *G-A* :: *word* $\Rightarrow$ τ_A $\Rightarrow$ *bool* and *G.lang*: $(\exists\, x.$ *G-S w x*$) \longleftrightarrow w \in$ *ilang G*.

3 Parser generation

parser : *local-theory* $\rightarrow$ *local-theory*



entry



parser *p* **for** *G* defines *p* :: *word* $\Rightarrow$ τ_S *option* for the start symbol S of G, together with soundness and completeness theorems. The optional first name selects the parser generator (by default *LR*) for all entries of the command.

parser *p* **for** *G* (A) parses from the nonterminal A of G instead: *p* :: *word* $\Rightarrow$ τ_A *option*, built from $G(\!|$*istart* := *''A''*$|\!)$, the grammar with the same productions and start A. Its theorems are those of the start symbol with A for S: the relational forms are about *G-A 0 w x* and the language forms

about *ilang* ($G(\!|istart := ''A''|\!)$), the language of A. (S) with S the start symbol is the plain form.

parser p **for** G **and** q **for** G (A) declares several parsers for one grammar, one after the other (parsers for another grammar need a command of their own). The entries share the grammar's decoders, its relation and the decoder and relation lemmas the theorems are discharged with, proved once for the command; each entry has its own tables, from its own start.

The only generator at present is *LR* (`lr/lr.ML`), described next.

4 LR parser

The definition is $p\ w$ = *Option.bind* (*dparse G Shn w*) *G--dec-S* (for an entry (A), *dparse* ($G(\!|istart := ''A''|\!)$) *Shn w* and *G--dec-A*): *dparse* is the verified driver of `lr/LR_Driver.thy`, an LALR(1)-style parser over the LR(0) automaton whose lookahead cells carry the layout constraints, *Shn* a fresh name for the augmented start symbol, and *G--dec-S* the decoder of the start symbol. For code generation the LR tables are constants *p-states*, *p-gtab*, *p-redtab*, *p-i0* and, for the driver, *p-xinit* and *p-xclos*, each defined by its term over the ones before it (*p-gtab* = *la-gtab G p-states*); generated code computes each once when it loads. The code equation of p refers to those constants, so neither *eval* nor exported code rebuilds the automaton per call. With **declare** [[*parser-literal-tables*]] the six constants are instead the evaluated tables, for exported code that must not compute them at startup; the command then evaluates the tables as well as its checks. If the grammar has no layout annotations, the static determinism check is empty and no lookahead cell of a reduce constrains the width of its token, the code equation instead runs the simple driver of `lr/LR_Simple.thy`, using the precise LR(0) automaton as well (tables *p-psts*, *p-pgt*, *p-pcomps*, *p-pfins*, *p-p0*). It keeps a stack of state indices and computes no closures at run time; *dparse-eq-sparse* proves that it computes the same result as *dparse*. Its code equation runs on indexed tables, computed from *p-ltabs*: productions and nonterminals are numbered, goto rows are keyed by terminal or by nonterminal number, and the reduces licensed at a state are tabulated by lookahead (*isctl-eq*). These tables are then stored as binary tries keyed by the bits of the index (`lr/LR_Trie.thy`, *jctl-eq*): they are static data, and with binary naturals a lookup involves no arithmetic, once the code equations *tlookup-binary* are declared next to `Code_Binary_Nat`. The input word is numbered once, pairing each token with its number in the tables (`lr/LR_Tok.thy`, *xrun-jctl2*). Finally, the reachable pairs of states of the two automata are numbered, *p-kpairs*, and the tables of the code, *p-ktabs*, are those of this product automaton, with one number per stack entry and the candidate reduces tabulated by lookahead (`lr/`

`LR_Prod.thy`, *xrun-kctl*); the closure of *p-kpairs* under shifts and gotos is checked by evaluation (*kok*). If the check fails, the code runs on the tries *p-jtabs*. On the product automaton the run detects loops of reductions by comparing with one saved stack instead of all stacks since the last shift (`lr/LR_Loop.thy`, *xrun-hist-brun*), and a step looks up one action table, *p-ptabs*, by pair number and lookahead: the shift target, or else the reduces with their productions (`lr/LR_Plain.thy`, *pctl-eq*). If the token type is a datatype whose constructors have no arguments, the number of a token is computed by a case distinction, *p-tnum*, instead of a search in the list of terminals; *pnum p-ptabs* = *p-tnum* is proved constructor by constructor. With **declare** [[*parser-int-tables*]] the tables are instead arrays with integer indices, *p-ztabs*, computed when the code is loaded (`lr/LR_Int.thy`, *zrun-eq*). For OCaml, `lr/LR_Simple.thy` implements immutable arrays (`iarray`) by OCaml arrays.

The facts *p.sound* and *p.complete* relate the parser to the language:

$$p.sound\text{: } p\ w = Some\ x \Longrightarrow w \in ilang\ G$$
$$p.complete\text{: } w \in ilang\ G \Longrightarrow \exists x.\ p\ w = Some\ x$$

For a grammar declared with (*relation*), *p.sound* and *p.complete* relate it to the relation of the start symbol:

$$p.sound\text{: } p\ w = Some\ x \Longrightarrow G\text{-}S\ 0\ w\ x$$
$$p.complete\text{: } G\text{-}S\ 0\ w\ x \Longrightarrow p\ w = Some\ x$$

The language forms are then available as *p.sound-lang* and *p.complete-lang*; should the bridge to the relation fail, the command warns and keeps the language forms under the plain names.

The generic theorems behind these facts carry premises that the command discharges when *p* is defined. Soundness needs only *fresh-name*, proved by simplification, and is therefore always unconditional.

Completeness needs in addition:

mdet G Shn, determinism of the driver on the grammar. The command evaluates the static report *rlalr-conflicts G Shn*, the LALR conflict analysis extended with the layout constraints. If it is empty, *mdet* follows by *rlalr-sufficient* and the command says so; otherwise it warns, lists the conflicting states, and keeps *mdet* as a premise of *p.complete*. The check is sufficient, not necessary: an ambiguous grammar always fails it, a deterministic one may fail it too. The parser is emitted either way; a run of it that revisits a configuration without consuming input (a unit self-loop whose reduce is licensed at the end of the input, say) ends with *None*.

Totality of the decoders on the parse trees of *G*.

For the relational forms, *acc-conflicts G Shn* = [] (again by evaluation).

Premises that cannot be discharged stay on the theorem as assumptions, so **thm** *p.complete* shows what has been established.

5 Example

A statement is an identifier or a *while* header followed by a block indented further than the header, all statements of a block aligned. Values count statements:

datatype *tok* = *While* | *Colon* | *Id*

grammar (*relation*) *py* =
  tokens ‹*tok*›
  terminals *while* = ‹*While*› **and** : = ‹*Colon*›
  productions
    Prog :: ‹*nat*› = *Block**
  and
    Block :: ‹*nat*›
      = *Stmt*$^=$
      | *Block*$^=$ *Stmt*$^=$ ⇒ ‹(+)›
  and
    Stmt :: ‹*nat*›
      = *Id*$^=$ ⇒ ‹λ-. *1*›
      | *while*$^=$ *Id*$^>$:$^>$ *Block*$^>$ ⇒ ‹λ- *n*. *n* + *1*›

parser *py-parse* **for** *py*

The token class *Id* is a value symbol, so the first action of *Stmt* takes the token record and ignores it. *Block** leaves the column of the outermost block open, *Block*$^>$ after a header demands a deeper one, and *Block*$^=$ *Stmt*$^=$ aligns the members.

The key theorem generated is

$$py.lang\text{: } (\exists\, x.\ py\text{-}Prog\ 0\ w\ x) = (w \in ilang\ py)$$

where *py-Prog* is the mutual **inductive** relation defined. For sake of brevity we omit the full *py.inducts*/*py.intros* and only show the intros for *py-Stmt*, writing $\langle t;\ sp\rangle$ for the token record $(\!|tok = t,\ span = sp|\!)$ (extension fields elided):

$$\frac{s\text{-}col\ sp = i}{py\text{-}Stmt\ i\ [\langle Id;\ sp\rangle]\ 1}$$

$$\frac{py\text{-}Block\ j\ w\ v \qquad s\text{-}col\ sp_1 = i \qquad i < s\text{-}col\ sp_2 \qquad i < s\text{-}col\ sp_3 \qquad i < j}{py\text{-}Stmt\ i\ (\langle While;\ sp_1\rangle\ \#\ \langle Id;\ sp_2\rangle\ \#\ \langle Colon;\ sp_3\rangle\ \#\ w)\ (v + 1)}$$

The static check is empty for *py*, so completeness is unconditional:

$$py\text{-}parse.complete\text{: } py\text{-}Prog\ 0\ w\ x \Longrightarrow py\text{-}parse\ w = Some\ x$$

With *tu c t* a token of class *t* at column *c*, the word *while x*: at column 1, *y* and *z* at column 3 and *w* at column 1 parses to *Some 4*, while moving *z* to

column 4 gives *None*:

lemma ‹*py-parse* [*tu 1 While*, *tu 7 Id*, *tu 8 Colon*, *tu 3 Id*, *tu 3 Id*, *tu 1 Id*] = *Some 4*›
by *eval*

lemma ‹*py-parse* [*tu 1 While*, *tu 7 Id*, *tu 8 Colon*, *tu 3 Id*, *tu 4 Id*] = *None*›
by *eval*

Further examples can be found in `tests/Regression_Tests.thy`.

References

[1] M. D. Adams. Principled parsing for indentation-sensitive languages: Revisiting Landin's offside rule. In *Proceedings of the 40th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2013)*, pages 511–522. ACM, 2013.